

SE3090 – SOFTWARE ENGINEERING FRAMEWORKS

Architecture Decision Record

Completed Example: CampusEats Restaurant Web Dashboard

ADR ID	ADR-001
DECISION	Use React for the restaurant web dashboard
STATUS	Accepted

Lecture 01: Framework Fundamentals, Selection and Emerging Trends

Professional lecture example and reusable student reference

ADR PURPOSE AND DOCUMENT OVERVIEW

Why this document matters

An Architecture Decision Record (ADR) captures an important technical decision, the evidence used to make it, the alternatives considered, and the consequences that follow. It prevents future teams from seeing only the final technology choice without understanding the reasoning behind it.

Traceability Connects the selected framework to project requirements, constraints and evidence.	Team alignment Creates one agreed record that developers, reviewers and future maintainers can follow.
Trade-off awareness Documents both benefits and limitations instead of presenting a technology as perfect.	Reviewability Defines the conditions under which the decision should be revisited or replaced.

Decision summary

ADR ID	ADR-001
Title	Use React for the CampusEats restaurant web dashboard
Status	Accepted
Decision date	13 July 2026
Decision owner	CampusEats Development Team
Related module topic	Framework selection, weighted decision matrix and decision recording

Key principle: The ADR must explain why the decision was made, not merely name the selected technology.

1

Context and problem statement

CampusEats is a food-delivery platform designed for university students and nearby restaurants. The product includes a student-facing mobile application, a restaurant web dashboard, a shared backend API, and a relational database.

The restaurant dashboard must support

- Restaurant profile, menu, price and availability management.
- Viewing, accepting or rejecting incoming orders.
- Updating order preparation and delivery status.
- Displaying daily and monthly sales information.
- Secure communication with the ASP.NET Core REST API.

Project constraints

TEAM	DEADLINE	BUDGET
3 developers	4 months	Limited
The team is too small to absorb a long framework-learning period.	The dashboard must reach a usable MVP quickly.	The project should rely mainly on mature open-source tooling.

2

Decision drivers

The team agreed to evaluate the candidates using the following weighted criteria.

Criterion	Weight	Why it matters
Team familiarity	30%	Reduces onboarding time and implementation risk.
Time to MVP	25%	Supports the fixed four-month delivery window.
Hiring pool	20%	Improves maintainability when the team grows.
Feature ecosystem	15%	Provides mature solutions for tables, forms and charts.
Long-term maintainability	10%	Supports upgrades, testing and future development.

3

Options considered

Option A — React	Strengths Strong team familiarity, large ecosystem, reusable components and a broad developer community.	Trade-offs Requires separate choices for routing, state management and project conventions.
Option B — Angular	Strengths Complete framework with strong conventions, routing, forms and dependency injection.	Trade-offs Higher learning curve and weaker fit with the short project deadline.
Option C — Vue	Strengths Gentle learning curve, lightweight architecture and good rapid-development support.	Trade-offs Lower team familiarity and a smaller hiring pool for this project context.

4

Weighted decision matrix

Scoring scale: 1 = very poor, 2 = poor, 3 = acceptable, 4 = good, 5 = excellent.

Criterion	Weight	React	Angular	Vue
Team familiarity	30%	4	2	3
Time to MVP	25%	4	3	5
Hiring pool	20%	5	3	3
Feature ecosystem	15%	5	4	4
Long-term maintainability	10%	4	5	4
Weighted total	100%	4.35	3.05	3.75

React total = $(0.30 \times 4) + (0.25 \times 4) + (0.20 \times 5) + (0.15 \times 5) + (0.10 \times 4) = 4.35 / 5$

5

Decision

ACCEPTED

The CampusEats development team will use React to build the restaurant web dashboard.

Rationale

- React achieved the highest weighted score: 4.35 out of 5.
- The team already has React experience, reducing training time and delivery risk.
- The ecosystem supports dashboard features such as forms, tables, charts and REST API integration.
- Reusable components support consistent design and incremental delivery.
- A broad developer community improves future recruitment and maintenance.

6

Consequences

Positive consequences

- Fast project start and shorter onboarding time.
- Reusable user-interface components.
- Strong support for API-driven applications.
- Good availability of documentation and packages.
- Easier recruitment of future developers.

Negative consequences

- Additional packages are required for routing and advanced state management.
- The team must define a consistent folder structure and coding conventions.
- Poorly maintained third-party packages may create upgrade risk.
- Package and framework versions require ongoing maintenance.
- React alone does not enforce a complete application architecture.

DECISION QUALITY CHECK: every selected technology must be justified using project-specific criteria, evidence and trade-offs.

7

Implementation guidance

The decision is implemented through the following minimum technical conventions.

Project structure	Use a feature-based folder structure with reusable components, pages and service modules.
Routing	Use React Router for authenticated dashboard navigation.
API communication	Create a dedicated API client and service layer for ASP.NET Core endpoints.
State management	Begin with local state and Context; adopt a larger state library only when justified.
Quality	Use linting, component tests and pull-request reviews before merging.
Security	Do not store sensitive tokens in source code; follow secure authentication and authorization practices.

8

Review conditions

The decision must be reviewed when one or more of the following conditions occur:

- A critical requirement cannot be implemented reliably using the selected React ecosystem.
- Required dependencies become unsupported, insecure or incompatible.
- Performance testing identifies a serious and persistent frontend limitation.
- The product grows into a large enterprise platform requiring stricter architectural conventions.
- The organisation adopts a mandatory frontend technology standard.

Approval and change history

Version	Date	Status	Owner / approver
1.0	13 Jul 2026	Accepted	CampusEats Development Team

REUSABLE STUDENT ADR TEMPLATE

Complete one ADR for each major architectural or framework decision. Keep the document concise, evidence-based and specific to your project.

ADR ID	<i>Click or type here</i>
Decision title	<i>Click or type here</i>
Status	<i>Click or type here</i>
Date / owner	<i>Click or type here</i>

1. Context

Describe the problem, users, requirements, constraints and why a decision is required.

2. Decision drivers

List the criteria used to compare the candidates. Add weights where appropriate.

3. Options considered

Describe at least two realistic alternatives, including their strengths and limitations.

4. Decision

State the selected option clearly and unambiguously.

5. Rationale

Explain why the selected option best fits the project evidence and criteria.

6. Consequences

Record positive outcomes, negative trade-offs, risks and new responsibilities.

7. Review conditions

State when the decision should be reviewed, replaced or superseded.

SE3090 | SELECT, JUSTIFY AND RECORD — DO NOT CHOOSE BY POPULARITY ALONE.